

k-NN y Perceptrón

Memorizar a los vecinos o resumir el mundo en un vector de pesos

Enrique Bolaños Abner Xocop

27 de agosto de 2026

Veritas · Libertas · Jvstitia

Agenda

- 1.** Dos filosofías
- 2.** k-NN desde cero
- 3.** La perilla k y la escala
- 4.** Maldición de la dimensionalidad
- 5.** El Perceptrón
- 6.** Convergencia y el muro del XOR
- 7.** Ejercicios
- 8.** Cierre

Dos formas opuestas de clasificar

Hoy vemos dos algoritmos que responden la misma pregunta (¿de qué clase es este punto?) con estrategias contrarias.

	k-NN	Perceptrón
Entrenamiento	guardar los datos	ajustar pesos
Memoria	todo el dataset	un vector w y un sesgo b
Decisión	local (los vecinos)	global (un hiperplano)
Costo	barato entrenar	barato predecir

Todo modelo elige qué recordar y qué olvidar

Predecir es mirar a los vecinos

Queremos clasificar granos de café de una finca de Huehuetenango como **especialidad** o **comercial**, con dos medidas: altura de cultivo y acidez.

El algoritmo completo, en tres pasos:

1. Calcular la distancia del punto nuevo a **todos** los puntos conocidos
2. Quedarse con los k más cercanos
3. Votar: la clase mayoritaria entre esos k gana

MÁS ALLÁ DEL CÓDIGO

No hay ecuación que aprender ni parámetros que estimar. La hipótesis de fondo es la localidad: puntos parecidos tienen etiquetas parecidas. Cuando esa hipótesis falla, k -NN falla.

La distancia, otra vez broadcasting

Distancia euclidiana entre un punto y n puntos: $d(x, p_i) = \sqrt{\sum_j (x_j - p_{ij})^2}$

```
1 import numpy as np
2 def distancias(X, punto):
3     # X: (n, d), punto: (d,)
4     dif = X - punto          # broadcasting (n,d)-(d,)
5     return np.sqrt((dif ** 2).sum(axis=1))
6 rng = np.random.default_rng(7)
7 X = rng.normal(size=(500, 2))
8 d = distancias(X, np.array([1.0, 1.0]))
9 d.shape                      # (500,)
```

Es exactamente el ejercicio 2 de la sesión 2: ya lo tenían escrito.

k-NN completo en 12 líneas

```
1 def knn_predict(X_train, y_train, X_test, k=5):
2     preds = []
3     for punto in X_test:
4         d = distancias(X_train, punto)
5         idx = np.argsort(d)[:k]          # k mas cercanos
6         vecinos = y_train[idx]
7         conteo = np.bincount(vecinos)
8         preds.append(conteo.argmax())    # voto mayoritario
9     return np.array(preds)
```

`np.argsort` ordena índices, no valores. Con datasets grandes conviene `np.argpartition`, que no ordena de más.

k es la perilla entre memorizar y promediar

- $k = 1$: cada punto de entrenamiento manda en su vecindad. Error de entrenamiento cero y frontera llena de islas: **overfitting**
- $k = n$: todos votan siempre lo mismo, la predicción es la clase mayoritaria global: **underfitting**
- Entre medio está el modelo útil

¿Cómo elegirlo? No con el set de prueba (eso es hacer trampa) sino con **validación cruzada**, que vemos en detalle el 17 de septiembre.

Dos costumbres útiles: probar k impar en clasificación binaria (evita empates) y arrancar cerca de $\sqrt{n}$.

k **controla la complejidad del modelo**

Escalar no es opcional en k-NN

Dos clientes descritos por (ingreso mensual en quetzales, años de antigüedad):

	Ingreso	Antigüedad
Cliente A	8000	2
Cliente B	8300	12

Distancia sin escalar: $\sqrt{300^2 + 10^2} \approx 300,2$. El ingreso aporta casi todo; la antigüedad (10 años de diferencia) es invisible.

Con `StandardScaler` las dos variables pesan lo mismo y la respuesta cambia.

Regla

La distancia hereda las unidades. Quien mide en unidades grandes se queda con el voto, aunque no sea el más informativo.

En dimensión alta, todos quedan lejos

```
1 for d in [2, 10, 100, 1000]:  
2     P = rng.normal(size=(1000, d))  
3     dif = P[:, None, :] - P[None, :, :]  
4     D = np.sqrt((dif ** 2).sum(axis=2))  
5     D = D[np.triu_indices(1000, k=1)]  
6     print(d, D.min() / D.max())
```

La razón entre la distancia mínima y la máxima se acerca a 1: el vecino más cercano y el más lejano dejan de distinguirse.

Si todas las distancias son casi iguales, el concepto de vecino se vacía de contenido y k-NN deja de tener sentido.

Lazy learning: el costo se paga después

k-NN no entrena: solo guarda. Toda la cuenta llega al predecir, $O(n \cdot d)$ por cada punto nuevo.

```
1 from sklearn.neighbors import KNeighborsClassifier
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4 modelo = make_pipeline(StandardScaler(),
5                        KNeighborsClassifier(n_neighbors=5))
6 modelo.fit(X_train, y_train)
7 modelo.score(X_test, y_test)
```

El Pipeline escala usando solo el train: así se evita leakage (sesión pasada). Volvemos a esto el 17 de septiembre.

1958: la primera neurona artificial

Frank Rosenblatt construye el Perceptrón con la idea de imitar una neurona: recibe señales, las pondera, y dispara si la suma supera un umbral.

$$z = w \cdot x + b = \sum_j w_j x_j + b$$
$$\hat{y} = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{si } z \leq 0 \end{cases}$$

Un modelo lineal seguido de un umbral. La frontera $w \cdot x + b = 0$ es una recta en 2D y un hiperplano en general.

MÁS ALLÁ DEL CÓDIGO

La prensa de la época anunció una máquina que pronto caminaría y hablaría. La distancia entre la demo y la promesa sigue viva en el campo.

La regla de aprendizaje

A diferencia de k-NN, aquí sí hay entrenamiento. Se recorren los ejemplos uno por uno (aprendizaje *online*):

1. Predecir $\hat{y}$ con los pesos actuales
2. Medir el error: $e = y - \hat{y}$, que solo puede ser 0, +1 o -1
3. Corregir: $w \leftarrow w + \eta e x$ y $b \leftarrow b + \eta e$

La lógica es directa: si acertamos, $e = 0$ y nada cambia. Si predijimos 0 cuando era 1, empujamos w **hacia** x para subir z la próxima vez.

Equivocarse mueve la frontera; acertar la deja quieta

Perceptrón desde cero

```
1 def perceptron(X, y, eta=0.1, epocas=50):
2     w = np.zeros(X.shape[1])
3     b = 0.0
4     for ep in range(epocas):
5         errores = 0
6         for xi, yi in zip(X, y):
7             pred = 1 if (w @ xi + b) > 0 else 0
8             e = yi - pred
9             w += eta * e * xi
10            b += eta * e
11            errores += abs(e)
12        if errores == 0:
13            return w, b, ep + 1
14    return w, b, epocas
```

El teorema de convergencia

Rosenblatt, 1962

Si los datos son **linealmente separables**, el perceptrón encuentra una frontera que los separa en un número finito de pasos.

Es una garantía fuerte y poco común: no depende del orden de los datos ni de la tasa η . Pero la letra pequeña importa:

- Si los datos **no** son separables, el algoritmo nunca se detiene y oscila para siempre
- Encuentra *alguna* frontera, no la mejor (esa idea del margen máximo lleva a SVM)
- No entrega probabilidades, solo un lado del hiperplano (la regresión logística arregla esto el 8 de septiembre)

El XOR: el muro de 1969

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

No existe una recta que deje los dos unos de un lado y los dos ceros del otro. El perceptrón no puede aprender XOR, y no por falta de datos ni de iteraciones: por su forma.

Minsky y Papert lo demuestran en 1969. El financiamiento se retira y empieza el llamado **invierno de las redes neuronales**.

La salida ya se conocía: apilar perceptrones en capas. Faltaba cómo entrenarlos, y eso llegó con backpropagation. Lo vemos el 6 de noviembre.

Dos fronteras, dos sesgos inductivos

	k-NN	Perceptrón
Frontera	irregular, a trozos	una recta o hiperplano
Supuesto	localidad	separabilidad lineal
Escalar	obligatorio	recomendable
Dimensión alta	se degrada rápido	la tolera bien
Ruido	sensible si k es chico	oscila si no separa

Cada modelo trae supuestos sobre cómo es el mundo. Ninguno es mejor siempre: el que gana es el que asume algo cierto sobre **sus** datos.

En el notebook graficamos las dos fronteras sobre el mismo dataset.

Ejercicio 1: su k-NN contra el de sklearn

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 X, y = load_iris(return_X_y=True)
4 X_tr, X_te, y_tr, y_te = train_test_split(
5     X, y, test_size=0.3, random_state=42)
6 # a) predecir con knn_predict (k=5) y medir accuracy
7 # b) comparar contra KNeighborsClassifier(5)
8 # ¿Que va aqui?
9 # Verificacion: deben coincidir casi exactamente
```

Ejercicio 2: la curva de accuracy vs k

```
1 ks = range(1, 41, 2)
2 acc_train, acc_test = [], []
3 for k in ks:
4     # ¿Que va aqui?
5     # entrenar, evaluar en train y en test, guardar
6     pass
7 # Graficar ambas curvas contra k
8 # Preguntas: donde esta el overfitting?
9 #           y el underfitting? que k eligen?
```

Ejercicio 3: contar las épocas del perceptrón

```
1 from sklearn.datasets import make_blobs
2 X, y = make_blobs(n_samples=200, centers=2,
3                   cluster_std=1.0, random_state=3)
4 # a) entrenar perceptron() y ver en cuantas épocas para
5 # b) subir cluster_std a 4.0 (clases traslapadas)
6 # ¿converge? ¿que pasa con los errores por época?
7 # c) probar con el XOR: X=[[0,0],[0,1],[1,0],[1,1]]
8 # ¿Que va aquí?
```

Resolvemos los tres en el notebook; lo que falte queda de práctica.

Lo esencial de hoy

- **k-NN**: no entrena, vota entre los k vecinos más cercanos
- k chico memoriza, k grande promedia; se elige con validación
- **Escalar es obligatorio**: la distancia hereda las unidades
- En **dimensión alta** todo queda equidistante y k-NN se degrada
- **Perceptrón**: modelo lineal con umbral; el error mueve los pesos
- Converge si los datos son separables; el **XOR** lo derrota

Próxima clase (martes 1 de septiembre): Lab Proyecto 1

Traigan su dataset y sus avances. **Parcial I: jueves 3 de septiembre** (Módulos 1 a 3)